Information Security

Lecture 3: Symmetric Cryptography

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

4 Symmetric Cryptography **3**
4.1 Cryptosystems, Kerckhoffs, Attack Models 3
4.2 Classical Ciphers and Their Cryptanalysis 4
4.3 XOR and the One-Time Pad 6
4.4 Confusion, Diffusion, Feistel 6
4.5 DES 7
4.6 AES 9
4.7 Modes of Operation 10
4.8 Key Exchange: Diffie–Hellman 12
Chapter takeaways 12

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 4

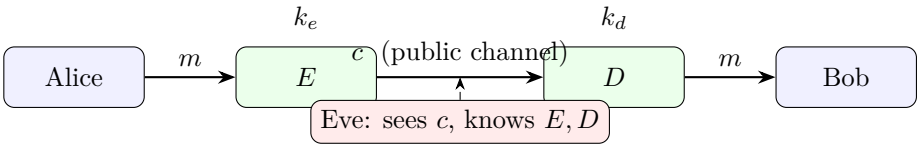
Symmetric Cryptography

4.1 Cryptosystems, Kerckhoffs, Attack Models

What are we trying to get?

We want the objects (message, key, cipher), the one design principle that governs all modern cryptography (Kerckhoffs), and the attack models in which security is stated; without a model, "this cipher is secure" is the under-defined circle question of Chapter 1.

Definition 4.1.1 (Cryptosystem)**.** *Cryptology* = *cryptography* (secret writing, hiding meaning) + *cryptanalysis* (breaking it). A cryptosystem is (M, C, K, E, D) with encryption $c = E(m, k_e)$ and decryption $m = D(c, k_d)$; *symmetric* if $k_e = k_d$ (everyone who knows k knows the whole secret), *asymmetric* if $k_e \neq k_d$ and k_d is infeasible to derive from k_e. The **key space** K is the set of keys; **brute force** tries them all. Cryptography provides *confidentiality* (against eavesdroppers), *integrity* (content unaltered) and *authentication* (origin as claimed); a cipher by itself gives only the first.



Key insight

Kerckhoffs's principle: the system must remain secure when everything except the key is public. Eve in the picture knows E and D. Consequences: algorithms are published and reviewed (DES and AES came out of open competitions); a compromised key is replaced without redesigning anything; a "secret algorithm" is a red flag because algorithms leak through reverse engineering and cannot be rotated. Historically: simple algorithms with long keys; today: sophisticated public algorithms with short keys.

Definition 4.1.2 (Attack models)**.** What the cryptanalyst holds besides the algorithm and the target ciphertext: **ciphertext only** (COA); **known plaintext** (KPA: some plaintext–ciphertext pairs under the same key); **chosen plaintext** (CPA: plaintexts of the attacker's choice get encrypted); **chosen**

ciphertext (CCA: ciphertexts of the attacker's choice get decrypted); **chosen text** (both). Power grows COA < KPA < CPA, CCA, and security under a stronger model implies the weaker ones. Four ways to attack: *analytical* (cleverness: a weakness in the algorithm or its use), *brute force* (money: parallel key search; *computationally secure* means the cost of cracking exceeds the value of the information), *social engineering* (the person), *implementation attacks* (software, hardware, side channels, usage).

Go deeper: the details an examiner asks about

- Modern ciphers must resist *chosen*-plaintext attacks because attackers routinely control inputs (a server encrypting their data); CPA security implies KPA and COA security.
- "Computationally secure" is economic (cost > value); "unconditionally secure" (the one-time pad) is information-theoretic.

Hard questions, answered

Q. A realistic setting for each attack model?
A. COA: intercepted radio traffic. KPA: encrypted files with known headers (PDF magic bytes). CPA: a web application that encrypts a cookie value the attacker supplies. CCA: a server that decrypts submitted tokens and reveals whether the padding was valid.

Q. Classify: power-trace analysis of a smart card; bribing the operator; trying all 2^{56} DES keys; exploiting ECB block independence.
A. Implementation attack; social engineering; brute force; analytical attack.

4.2 Classical Ciphers and Their Cryptanalysis

What are we trying to get?

Three historical ciphers teach three lessons: a tiny key space falls to brute force (Caesar); a huge key space is not enough when statistics survive (substitution); a repeating key reduces to the previous case (Vigenère). We prove the second and give the algorithm for the third.

Definition 4.2.1 (Caesar, substitution, Vigenère)**. Caesar**: $c = (p + k) \bmod 26$ on letters 0..25; key space 26 (`Security is very interesting` with $k = 3 \to$ `VHFXULWB LU YHRB LQWHUHVWLQJ`). **Simple substitution**: a fixed permutation σ of the alphabet, $c = \sigma(p)$; key space $26! \approx 4.03 \times 10^{26}$. **Vigenère**: a keyword $k_0 \dots k_{L-1}$ repeats and letter i is shifted by $k_{i \bmod L}$ (`send reinforcements` under `somesecret` $\to$ `kszhjikejhjqqqwrvj`).

Theorem 4.2.2 (Substitution preserves the frequency histogram)**.** *Under a monoalphabetic substitution, the ciphertext letter counts are a relabelling of the plaintext letter counts. Hence the key is recovered letter by letter from language statistics, in time independent of the key-space size.*

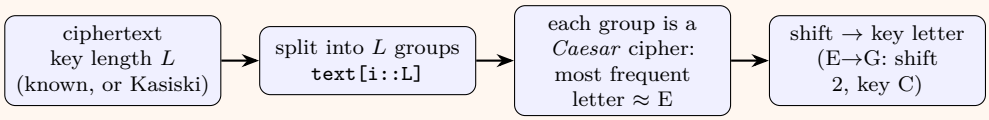
Proof. For each plaintext letter a, $\#\{i : c_i = \sigma(a)\} = \#\{i : p_i = a\}$ because σ

is a bijection applied position-wise. The multiset of counts is unchanged; only the labels move. English counts are far from uniform (E 12.7%, T 9.1, A 8.2, O 7.5, I 7.0, N 6.7, S 6.3, H 6.1, R 6.0, D 4.3, L 4.0, C 2.8; J, X, Q, Z rarest), so sorting the ciphertext counts largely determines σ; short words, doubled letters and repeated patterns resolve the remaining ambiguity. □

Key insight

A large key space is necessary, not sufficient. Brute force is defeated by 26!; analysis is not, because the cipher leaves the *structure* of the plaintext intact. Every modern design goal (confusion, diffusion, §4.4) exists to destroy exactly this structure.

Technique: Breaking Vigenère



Every letter in group i was shifted by the same k_i, so Theorem 4.2.2 applies within the group with the simplest possible σ. Unknown L: repeated trigrams in the ciphertext occur at distances that are multiples of L (Kasiski), or compare the index of coincidence for candidate lengths. The attack needs the key to *repeat*: a random key as long as the message, never reused, leaves one letter per group and no statistics (§4.3).

Worked example: The TP escape room

Caesar $k = 4$: `CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ` → `GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD`. Decrypt with $k = 8$: `GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA` → `YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS`. Unknown key, try all 26: `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP` is shift 11, `THE KEY IS UNKNOWN TRY BRUTE FORCE`. Substitution table (A→Q, B→M, C→J, D→Z, E→T, ...): `KEEP THE SECRET SAFE` → `LTTN EKT YTJRTE YQGT`; `ERVYE DXVR SXFPJ` → `TRUST YOUR LOGIC`. The slide's `GSRH RH Z HVXIVG` is the reversed alphabet (Atbash): `THIS IS A SECRET`.

Go deeper: the details an examiner asks about

- Enigma gave the German navy an early advantage in WWII; broken with Turing's *bombe*. The lesson the course draws: with computers, substitution and transposition alone are obsolete; modern ciphers combine substitution with repeated transposition so that ciphertext looks random (DES, AES).
- The TP hint order for its specific text, E > T > R > S/O > A > H > N > D > I > G > L, shows that frequency analysis gives a *hypothesis*; context finishes the job.

Hard questions, answered

Q. Why is a substitution cipher with 26! keys broken "faster" than Caesar is brute-forced?
A. Both are trivial, but for different reasons: Caesar by 26 trials, substitution by analysis of surviving statistics. The point is that 26! bought no security because the attack never enumerates keys.

Q. Vigenère with a 1000-letter random key on a 1000-letter message, used once. Breakable by the group method?
A. No: $L = 1000$, one letter per group, no statistics; that is the one-time pad. The attack needs repetition.

4.3 XOR and the One-Time Pad

XOR is "different $\to$ 1, same $\to$ 0", and $(x \oplus k) \oplus k = x$, so one operation both encrypts and decrypts. `Hello` $\oplus$ the repeating key `key` gives `00100011 00000000 00010101 00000111 00001010`; XOR with the key again returns `Hello`.

Theorem 4.3.1 (One-time pad)**.** *If the key is uniformly random, as long as the message, and used once, then for every ciphertext c and every candidate plaintext m of that length there is exactly one key with $m \oplus k = c$; a ciphertext-only attacker learns nothing about m.*

Proof. $k = m \oplus c$ is the unique key; since all keys are equally likely, all plaintexts of that length are equally consistent with c. □

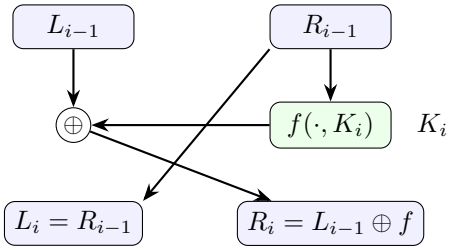
With a *repeating* key the scheme is Vigenère on bytes and fails in every model: KPA gives $m \oplus c = k$ directly; CPA with an all-zero plaintext returns k as the ciphertext; CCA with an all-zero ciphertext returns k as the "plaintext"; COA notices that the second ciphertext byte is `00000000`, hence the second key byte equals the second plaintext byte, and proceeds from language structure. Reuse is the fatal case: $c_1 \oplus c_2 = m_1 \oplus m_2$ and two structured texts can be pulled apart (crib dragging).

Hard questions, answered

Q. Why is the one-time pad rarely used although it is perfectly secure?
A. The key must be as long as all traffic, truly random, delivered secretly, and never reused; the key-distribution problem is as hard as the message-distribution problem. Block ciphers use a short key to *generate* a long pseudorandom stream (CTR mode) instead.

4.4 Confusion, Diffusion, Feistel

Definition 4.4.1 (Confusion and diffusion (Shannon))**. Confusion**: make the relation between key, plaintext and ciphertext complex (nonlinear); Caesar's $c = (p+k) \bmod 26$ has none. **Diffusion**: spread each input bit's influence over many output bits so that local statistics disappear. Block ciphers obtain both by iterating *rounds* that substitute (confusion) and permute or mix (diffusion).



A **Feistel round**: only the right half enters f; the left half is XOR-ed with the result; the halves swap. Decryption runs the same round with the same K_i because XOR cancels.

Theorem 4.4.2 (Feistel networks invert for free)**.** *For any function f, the round $(L_{i-1}, R_{i-1}) \mapsto (L_i, R_i)$ with $L_i = R_{i-1}$ and $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$ is inverted by $R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$. An n-round Feistel cipher is therefore decrypted by the encryption algorithm with the subkeys in reverse order, and f need not be invertible.*

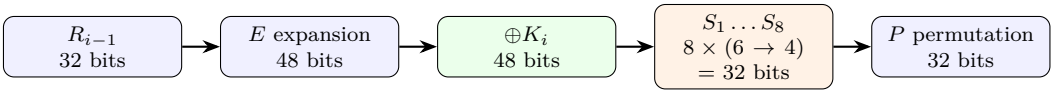
Proof. L_i is a copy of R_{i-1}, so $f(R_{i-1}, K_i) = f(L_i, K_i)$ is recomputable from the output; XOR-ing it into R_i cancels it since $x \oplus y \oplus y = x$. Apply from round n down to 1. □

4.5 DES

What are we trying to get?

DES is the worked example of a Feistel cipher: we want its parameters, the four steps of its round function with one round computed by hand, the key schedule and why reversing it decrypts, and why it fell.

Definition 4.5.1 (DES)**.** Block 64 bits, key 56 bits (the 64-bit input key carries 8 parity bits at positions 8, 16, …, 64, dropped by permuted choice PC-1), 16 Feistel rounds with distinct 48-bit subkeys, an initial permutation IP and its inverse IP^{-1} at the ends. The round function $f(R, K_i)$, 32 bits in and out: (1) **expansion** E, 32 → 48 bits, by duplicating the two boundary bits of each 4-bit group; (2) XOR with K_i; (3) eight **S-boxes** $S_1..S_8$, each 6 bits → 4: the *row* is the outer bits b_1b_6, the *column* the inner bits $b_2b_3b_4b_5$; the only nonlinear step and "crucial for DES security"; (4) **permutation** P, a fixed rearrangement sending each S-box's output bits to different S-boxes in the next round (diffusion). IP, E and P are position tables: they move bits and never change their values.



K_i from the key schedule

Worked example: One round: the slide's numbers and the toy round

Slide: input `0x0123456789ABCDEF`; IP's first row (58, 50, 42, 34, 26, 18, 10, 2) selects input bits that form the first output byte `11001100`. After IP, $R_0 =$ `0xF0AAF0AA`; $E(R_0) =$ `011110 100001 010101 010101 011110 100001 010101 010101`; after $\oplus K_1$: `0x6117BA866527`; the first 6-bit group `011000` has row 00 = 0 and column 1100 = 12, and $S_1[0, 12] = 5 =$ `0101`; all eight boxes give `0x5C82B597`; P yields `0x234AA9BB`. Toy round ($L_0 =$ `1010`, $R_0 =$ `1001`, $E' = [4, 1, 2, 3, 4, 1]$, $K_1 =$ `101011`, $P' = [2, 4, 3, 1]$): $E'(R_0) =$ `110011`; $\oplus K_1 =$

011000; $S_1 \to$ 0101; $P' \to$ 1100; $L_1 = R_0 =$ 1001; $R_1 = L_0 \oplus$ 1100 = 0110.

Definition 4.5.2 (Key schedule; decryption)**.** PC-1 keeps 56 bits and splits them into 28-bit halves C_0, D_0. In round i each half is rotated left by 1 bit in rounds 1, 2, 9, 16 and by 2 bits in all other rounds; PC-2 then selects a permuted 48-bit subset as K_i. The total rotation is $4 \cdot 1 + 12 \cdot 2 = 28$, so $C_{16} = C_0$ and $D_{16} = D_0$. Decryption uses the same 16 keys in order $K_{16} \dots K_1$ (Theorem 4.4.2); equivalently, run the schedule with no rotation in round 1, one bit to the *right* in rounds 2, 9, 16 and two bits right elsewhere. No new keys are generated.

Worked example: Toy key schedule

Key 10101101; PC-1 $= [1,2,3,5,6,7]$ keeps 101110; $C_0 =$ 101, $D_0 =$ 110. Round 1, shift 1: $C_1 =$ 011, $D_1 =$ 101; PC-2 $= [1,2,4,5]$ on 011101 gives $K_1 =$ 0110. Round 2 continues from C_1, D_1 (never restart from the master key): $C_2 =$ 110, $D_2 =$ 011; 110011 $\to K_2 =$ 1101.

Interpretation & discussion

Strength: $2^{56} \approx 7.2 \times 10^{16}$ keys. At 10^9 keys per second one machine needs 7.2×10^7 s ≈ 2.3 years; the EFF's $250,000 machine (1998) cracked DES in under two days. DES died of key size, not of cryptanalysis. **3DES** applies DES three times in encrypt–decrypt–encrypt order (so $K_1 = K_2 = K_3$ degrades to DES for compatibility) with two keys (112 bits effective) or three (168). *Double* DES is pointless: *meet in the middle* encrypts a known plaintext under all 2^{56} values of K_1, decrypts its ciphertext under all 2^{56} values of K_2, and matches the tables, for $\approx 2^{57}$ work instead of 2^{112}. 3DES survives in legacy finance (ATMs); AES replaced it and runs 5× faster in CPU hardware.

Go deeper: the details an examiner asks about

- Why DES decrypts with reversed keys and AES does not: Theorem 4.4.2 needs the Feistel structure; AES acts on the whole block and needs explicit inverse operations.
- S-box indexing trap: "outside bits = row, inside bits = column". Row 2 of S_1: 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0.
- Timeline: DES 1975 (56/64); 3DES 1995 (112/168, 64); Twofish, Serpent, Rijndael 1998 (128/192/256, 128); AES standard 2001 (FIPS-197).

Hard questions, answered

Q. S_1 on 100110?
A. Row $b_1 b_6 =$ 10 $= 2$, column 0011 $= 3$: $S_1[2,3] = 8 =$ 1000.

Q. Why 56 key bits from a 64-bit input, and why is 2DES only ≈ 57 bits strong?
A. Eight parity bits are discarded by PC-1. Meet in the middle replaces one 2^{112} search by two 2^{56} tables.

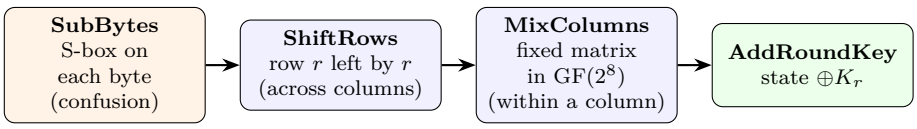
4.6 AES

What are we trying to get?

AES is the substitution–permutation design: we want the byte arithmetic it lives in, its four round operations with the diffusion they produce, one MixColumns byte by hand, the key expansion, and the decryption order; the TP's "AES without diffusion" shows why each piece is needed.

Definition 4.6.1 (GF(2^8) arithmetic)**.** Bytes are polynomials of degree < 8 with coefficients in $\{0, 1\}$. Addition is XOR. Multiplication is polynomial multiplication reduced modulo $m(x) = x^8+x^4+x^3+x+1$ (`0x11B`). Multiplying by x (`02`, `xtime`): shift left one bit, and if the bit shifted out was 1, XOR with `1B` (because $x^8 \equiv x^4 + x^3 + x + 1$). $03 \cdot b = (02 \cdot b) \oplus b$. Every nonzero byte has an inverse, which is what makes SubBytes and MixColumns invertible.

Definition 4.6.2 (AES-128)**.** Rijndael (Rijmen and Daemen; FIPS-197, 2001). Block 128 bits held as a 4×4 byte *state* filled column by column (b_{i+4j} at row i, column j); keys of 128/192/256 bits with 10/12/14 rounds and $N_r + 1$ round keys; *not* Feistel. Encryption: AddRoundKey(K_0); rounds $1..N_r - 1$ of **SubBytes** (each byte through the S-box; confusion), **ShiftRows** (row r rotated left by r; moves bytes *between* columns), **MixColumns** (each column multiplied by the fixed matrix in GF(2^8); mixes *within* a column), **AddRoundKey** (XOR with K_r); a final round without MixColumns. ShiftRows and MixColumns together give diffusion: after two rounds every byte depends on every input byte. Decryption: AddRoundKey(K_{10}); for rounds 9..1: InvShiftRows (right rotations), InvSubBytes (inverse S-box), AddRoundKey, InvMixColumns (matrix `0E 0B 0D 09`); finally InvShiftRows, InvSubBytes, AddRoundKey(K_0).



one round; N_r = 10/12/14 rounds; AddRoundKey(K_0) comes first; the last round skips MixColumns

Worked example: MixColumns, first byte of column `D4 BF 5D 30`

$b_0 = (02 \cdot \text{D4}) \oplus (03 \cdot \text{BF}) \oplus 5\text{D} \oplus 30$. $02 \cdot$ D4: `11010100` $\rightarrow$ `10101000` = `A8`, a 1 was shifted out, so $\oplus$`1B` = `B3`. $02 \cdot$ BF: `10111111` $\rightarrow$ `01111110` = `7E`, carry, $\oplus$`1B` = `65`; $03 \cdot$ BF = 65 $\oplus$ BF = DA. Then B3 $\oplus$ DA = 69, 69 $\oplus$ 5D = 34, 34 $\oplus$ 30 = 04. The full column becomes `04 66 81 E5`; AddRoundKey with `A0` gives `A4`. Other slide values: SubBytes `19`$\rightarrow$`D4`, `00`$\rightarrow$`63`, `03`$\rightarrow$`7B`, `71`$\rightarrow$`A3`; ShiftRows on the index state: row 1 `11 12 13 10`, row 2 `22 23 20 21`, row 3 `33 30 31 32`; AddRoundKey with the all-`0F` key XORs each byte with `0F` (`23`$\rightarrow$`2C`).

Definition 4.6.3 (Key expansion (AES-128))**.** 44 four-byte words; $w_0..w_3$ are the key. For $i \geq 4$: $t = w_{i-1}$; if $i \equiv 0 \pmod 4$, $t \leftarrow$ RotWord (rotate one byte), SubWord (S-box each byte), XOR `Rcon`$[i/4]$ into the first byte; then

$w_i = w_{i-4} \oplus t$. Round key r is $w_{4r}..w_{4r+3}$. `Rcon` first bytes: `01 02 04 08 10 20 40 80 1B 36`, the successive powers of `02` in the field (hence `1B` after `80`).

From the TP: AES without diffusion

Remove ShiftRows and MixColumns from a two-round cipher on one byte: $c = S(S(p \oplus k_0) \oplus k_1) \oplus k_2$. From the pair $(00, \text{A4})$, $k_2 = \text{A4} \oplus S(S(k_0) \oplus k_1)$ is a function of (k_0, k_1); enumerate the 2^{16} pairs, derive k_2, keep those consistent with $(01, \text{F8})$ and $(02, \text{3F})$. Each byte position is attacked *independently* with three known plaintexts; with diffusion the 16-byte state must be attacked as a whole (2^{128}). Confusion without diffusion is attackable byte by byte.

Go deeper: the details an examiner asks about

- The last round omits MixColumns because a trailing public linear map adds no security and would break the symmetry between encryption and decryption.
- SubBytes is the only nonlinear step; without it AES is a linear map, solvable by linear algebra from a few plaintext–ciphertext pairs.
- AES-128 vs AES-256: key length, rounds (10 vs 14), round keys (11 vs 15), expansion details; block size and the four operations are unchanged.

Hard questions, answered

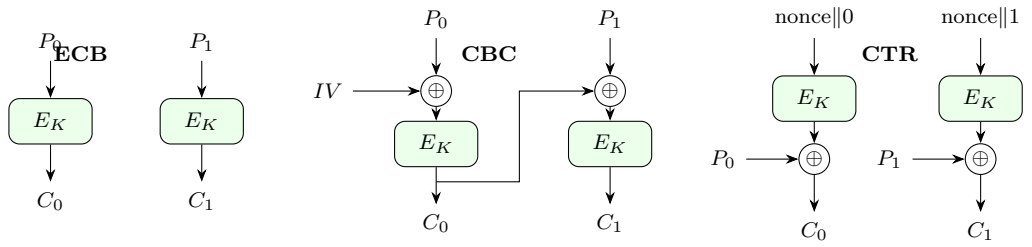
Q. Compute $02 \cdot 57$ and $03 \cdot 57$.
A. `57` = `01010111` → `10101110` = `AE`, no carry: $02 \cdot 57 = \text{AE}$; $03 \cdot 57 = \text{AE} \oplus 57 = \text{F9}$.

Q. Which step gives confusion, which give diffusion, and what breaks without each?
A. SubBytes: confusion (remove it: linear, trivial). ShiftRows + MixColumns: diffusion (remove them: the TP's byte-wise 2^{16} attack). AddRoundKey injects the key (remove it: no secret at all).

4.7 Modes of Operation

What are we trying to get?

A block cipher encrypts one block; a mode says how to encrypt a message of many blocks. We want the three modes of the course with their equations, their parallelism and error behaviour, and the one visual fact every student must be able to explain: why ECB shows the penguin.



Definition 4.7.1 (ECB, CBC, CTR)**.** **ECB**: $C_i = E_K(P_i)$; parallel both ways; deterministic per block, so equal plaintext blocks give equal ciphertext blocks and the *structure* of the data leaks. **CBC**: $C_i = E_K(P_i \oplus C_{i-1})$ with $C_0 = IV$, and $P_i = D_K(C_i) \oplus C_{i-1}$; encryption is sequential (needs C_{i-1}), decryption parallel with random access (needs only C_i, C_{i-1}); the **IV** is random and *not secret* (sent in clear); it prevents a repeated first block from producing a repeated ciphertext. **CTR**: $C_i = P_i \oplus E_K(\text{nonce}\|i)$, a stream cipher built from a block cipher, fully parallel, random access; the nonce must never repeat under one key (two-time pad). Others: CFB, OFB, and CCM = CTR for confidentiality plus CBC-MAC for integrity (authenticated encryption).

Theorem 4.7.2 (Error propagation in CBC)**.** *A flipped bit in ciphertext block C_i garbles plaintext block P_i entirely and flips exactly that bit in P_{i+1}; all later blocks decrypt correctly.*

Proof. $P_i = D_K(C_i) \oplus C_{i-1}$ pushes the corrupted block through D_K, whose output is pseudorandom. $P_{i+1} = D_K(C_{i+1}) \oplus C_i$ XORs the corrupted C_i directly, bit for bit. P_{i+2} depends only on C_{i+1}, C_{i+2}. □

From the TP: OpenSSL mission 2: ECB on an image

OpenSSL's salted format starts with the 8-byte signature `Salted__` followed by an 8-byte *salt* (public; the key is derived from password and salt with PBKDF2), so the first 16 bytes are not ciphertext. The image is $432 \times 475 = 205{,}200$ bytes, exactly 12,825 blocks, so PKCS#7 appends one full padding block: ciphertext 205,216 bytes, file 205,232. Counting equal 16-byte blocks (`collections.Counter`) and giving equal blocks the same grey shade reproduces the picture's outline: a *pattern, not the secret*, since the pixel values stay hidden. Decrypt with `openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw`; re-encrypting with `-aes-256-cbc` leaves essentially no repeated blocks because each block is XOR-ed with the previous ciphertext first.

Go deeper: the details an examiner asks about

- A random IV makes encryption *probabilistic*: the same message twice gives different ciphertexts; a fixed IV reintroduces an ECB-like leak for the first block.
- CBC without a MAC allows bit-flipping attacks on P_{i+1} by Theorem 4.7.2: confidentiality is not integrity (Chapter 5).

Hard questions, answered

Q. Why is a clear-text IV not a weakness, and what would be?
A. The IV only needs to be unpredictable or unique so that equal first blocks differ; knowing C_0 does not help invert E_K. Reusing an IV, or using a predictable one under CPA, is the weakness.

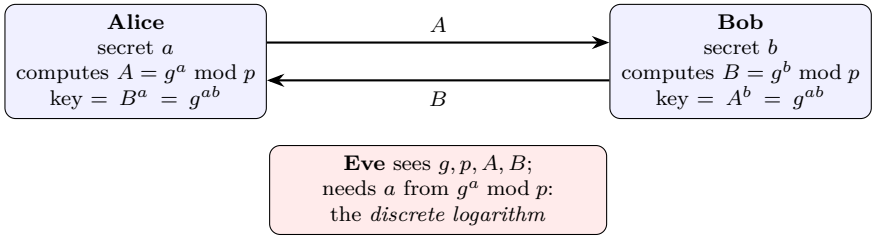
Q. Rank ECB, CBC, CTR on encryption parallelism / decryption parallelism / random access / pattern hiding.
A. ECB yes/yes/yes/no; CBC no/yes/yes/yes; CTR yes/yes/yes/yes (with a

unique nonce).

Q. Strong cipher, weak system: AES-256-ECB on a column of 16-byte customer IDs.
A. Equal IDs give equal ciphertexts; frequency and equality analysis works without touching AES. Use CBC or CTR with fresh IVs, or authenticated encryption.

4.8 Key Exchange: Diffie–Hellman

Symmetric ciphers need the key delivered over a secure channel. Ralph Merkle proposed public-key ideas as a Stanford student in 1974 (shot down by his professor) and in 1975 (rejected by CACM as "extremely dangerous to transmit key information in the clear"); Diffie and Hellman published in 1976.



Public g and p; both compute the same $g^{ab} \bmod p$ from what they received and their own secret. Eve must solve the **discrete logarithm** (slide: $6^x \bmod 13 = 2$, so $x = 5$; $6^x \bmod 13 = 9$, $x = 4$), trivial for $p = 13$, infeasible for a 2048-bit p. DH yields a shared secret but *no authentication*: a **man in the middle** runs one exchange with Alice (key k_1) and another with Bob (key k_2) and relays, reading and altering everything ("Transfer \$10" → "Transfer \$100"). The endpoints must be authenticated, which is where Chapter 5 begins.

Hard questions, answered

Q. Which hard problem underlies DH and which underlies RSA?
A. Discrete logarithm; integer factorisation.

Q. Does encrypting the DH messages with Bob's public key stop the MITM?
A. Only if Alice already holds Bob's *authentic* public key; if Eve can substitute hers, she decrypts and relays. The issue is key authenticity, hence certificates.

Chapter takeaways

Summary: Symmetric cryptography

- Kerckhoffs: only the key is secret. Models COA < KPA < CPA/CCA; attack by analysis, brute force, people or implementation.
- Substitution falls to frequency analysis (Theorem 4.2.2); Vigenère splits into L Caesar groups; a non-repeating random key is the one-time pad (Theorem 4.3.1).
- Confusion (nonlinear S-boxes) and diffusion (permutation, mixing); Feistel inverts for free with reversed keys (Theorem 4.4.2).

- DES: 64/56, 16 rounds of E–XOR–S-box–P, rotations 1 in rounds 1, 2, 9, 16; brute-forced in 1998; 2DES falls to meet in the middle.
- AES: GF(2^8) with `1B`; SubBytes, ShiftRows, MixColumns, AddRoundKey; 10/12/14 rounds; SPN needs explicit inverses.
- ECB leaks structure; CBC chains with a public random IV; CTR is a parallel stream; CCM adds integrity.
- DH: discrete logarithm; shared secret without authentication, hence MITM.